

**Luiss**

Libera Università Internazionale  
degli Studi Sociali Guido Carli

# Hardware

## Introduction to Computer Programming

**Management and Artificial Intelligence**

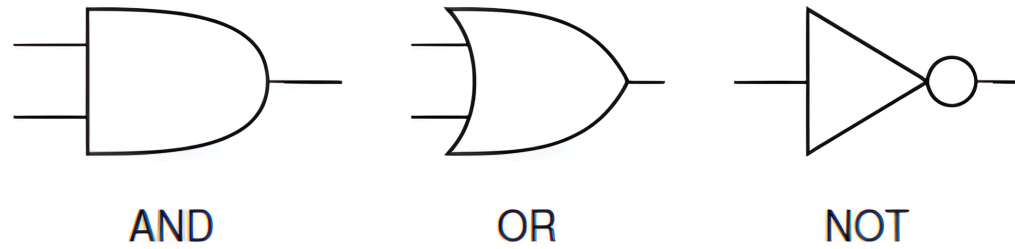
LUISS



# Boolean operators as logic gates

The boolean operators AND, OR, and NOT, introduced in the previous lecture, can be implemented as logic gates (circuits).

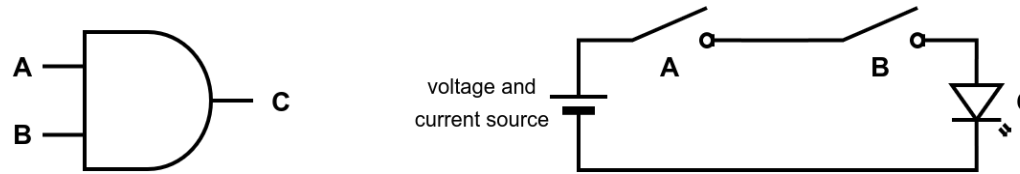
Within a circuit, they are represented by these symbols:



| A | B | A AND B | A OR B | NOT A |
|---|---|---------|--------|-------|
| 0 | 0 | 0       | 0      | 1     |
| 0 | 1 | 0       | 1      | 1     |
| 1 | 0 | 0       | 1      | 0     |
| 1 | 1 | 1       | 1      | 0     |

# AND gate

The AND logic gate can be implemented with two switches in series:



The two inputs, A and B, control the two switches:

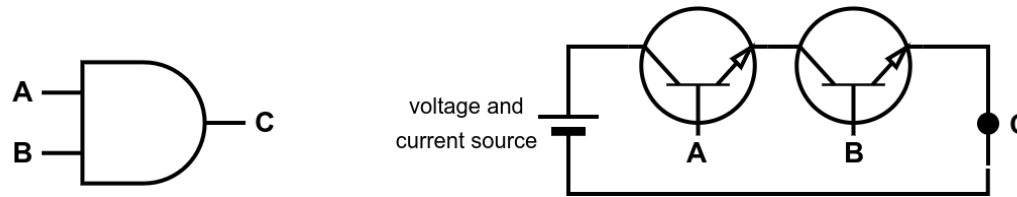
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

Only when both switches are closed (A=True and B=True), the current flows to C (represented with a LED, which would emit light to represent a True value).

**NOTE:** A resistor should be added in series with the LED to limit the current and prevent damage.

# AND gate: from switches to transistors

In electronics, when realizing an AND gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

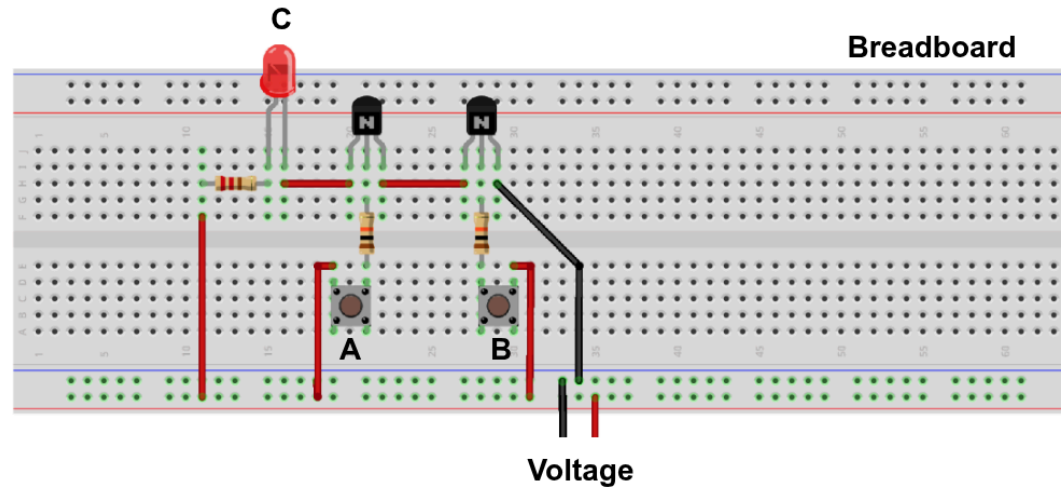
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

Only when both transistors are on, the output signal C is high.

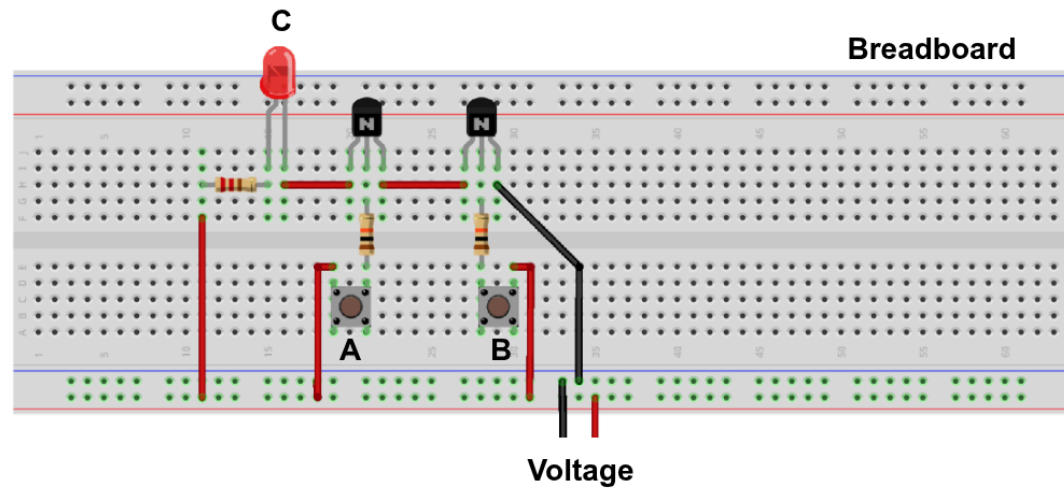
**NOTE:** There exist other types of transistors, which can be used to implement an AND gate.

# AND gate: real-world (inefficient) implementation

We can buy transistors, resistors, and LED to make our own AND gate with minimal cost: "electronics starter kit" (often sold with Arduino boards) cost less than 15-20€ and allows to build simple circuits.



- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if and only if both A and B are pressed

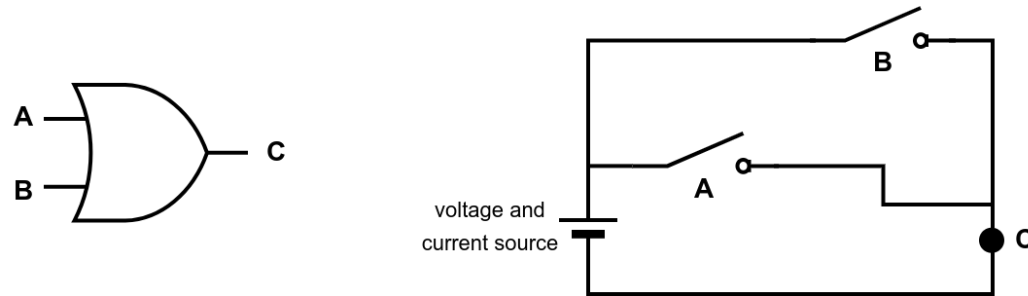


In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to control another circuit or to generate a bit value

# OR gate

The OR logic gate can be implemented with two switches in parallel:



The two inputs, A and B, control the two switches:

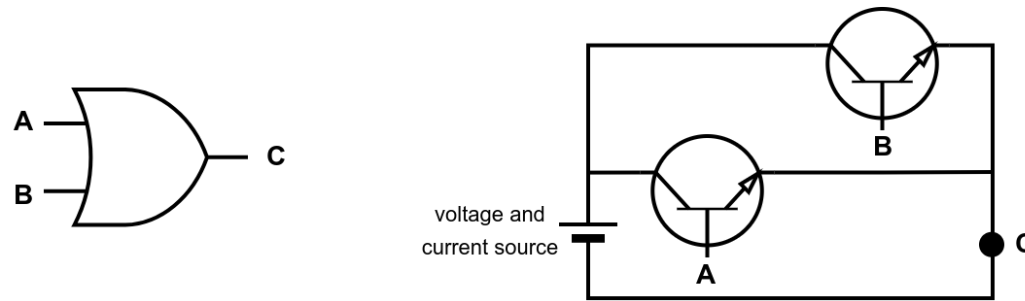
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

When at least one switch is closed ( $A=\text{True}$  or  $B=\text{True}$ ), the current flows to C (represented with a LED, which would emit light to represent a True value).

**NOTE:** A resistor should be added in series with the LED to limit the current and prevent damage.

# OR gate: from switches to transistors

In electronics, when realizing an OR gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

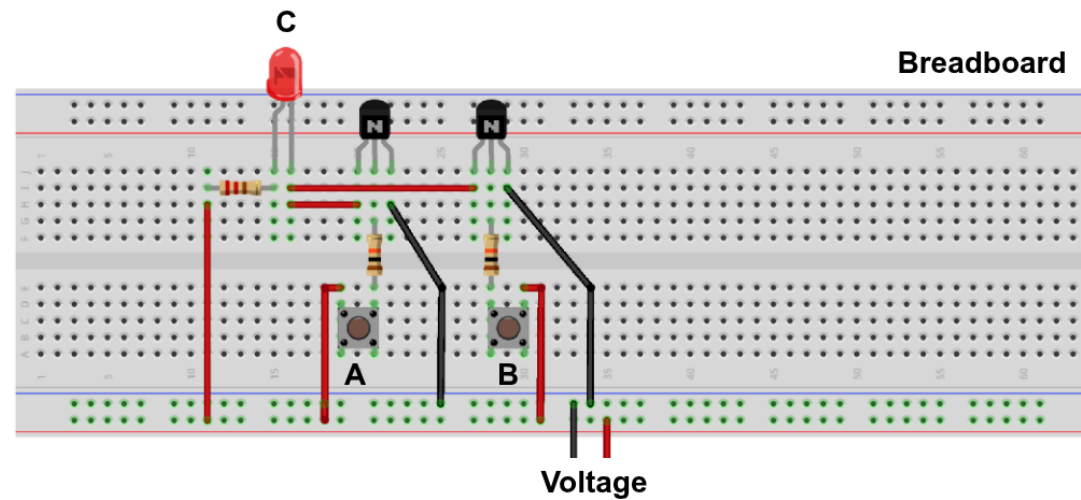
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

When at least one transistor is on, the output signal C is high.

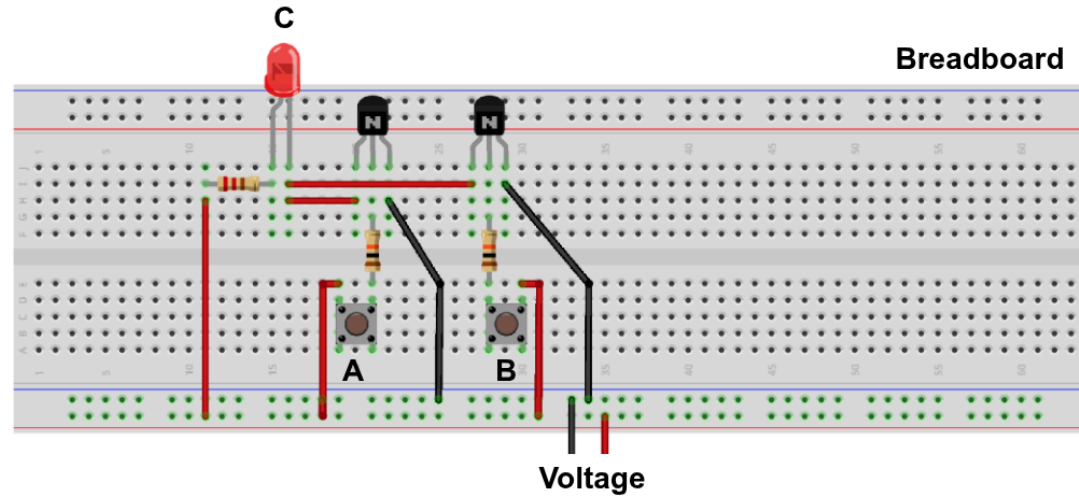
**NOTE:** There exist other types of transistors, which can be used to implement an OR gate.

# OR gate: real-world (inefficient) implementation

As for the AND gate, we can implement the OR gate with a few cheap transistors, resistors, and LEDs.



- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if at least one of A or B is pressed



In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to control another circuit or to generate a bit value

# What can we do with AND, OR, NOT gates?

We can combine AND, OR, and NOT gates to implement a logic circuit that implements arbitrary boolean functions:

1. Build the truth table for your custom computation. For instance:

| A | B | C | OUTPUT |
|---|---|---|--------|
| 0 | 0 | 0 | 1      |
| 0 | 0 | 1 | 1      |
| 0 | 1 | 0 | 0      |
| 0 | 1 | 1 | 1      |
| 1 | 0 | 0 | 1      |
| 1 | 0 | 1 | 1      |
| 1 | 1 | 0 | 0      |
| 1 | 1 | 1 | 1      |

2. Use a tool to derive the boolean formula: e.g., provide the OUTPUT column from your truth table to [dcode.fr](https://dcode.fr/boolean-expressions-calculator)

See also: **boolean Expressions Calculator – boolean Minterms and Maxterms**

### FIND EQUATION FROM TRUTH TABLE

Indicate only the output values of the function (the last column from the boolean truth table)

★ OUTPUT VALUES (LIST OF 0 AND 1)

11011101

★ TABLE ORDERED (INPUT VALUES) ☒ FROM 0,...,0 TO 1,...,1  
☐ FROM 1,...,1 TO 0,...,0

★ BOOLEAN NOTATION ☒ LITERAL (AND, OR, NOT)  
☐ LOGICAL ( $\wedge$ ,  $\vee$ ,  $\neg$ )  
☐ PROGRAMMING (&&, ||, ~)  
☐ ALGEBRAIC (\*, +, !)

▶ CALCULATE

Results

f(a,b,c)=

NOT b OR c

**NOTE:** We discovered that our output does not depend on the value of A!

3. Use a tool to get a minimal circuit: e.g., provide the boolean formula to [boolean-algebra.com](https://boolean-algebra.com)

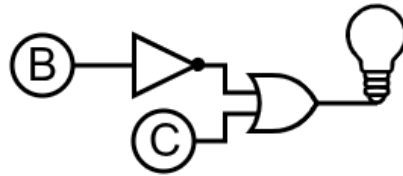
## Boolean Algebra Simplifier

Help: Press '!' to insert a Not

Solution:  $\overline{B}+C$

## Logic Gate

View Non-minimized



**NOTE:** In this case, the formula was already minimal. In general, the tool will first perform a minimization of the formula, and then generate the circuit. The smaller the formula, the smaller the circuit; fewer gates, less area, less power, lower cost.

# Why do we care about equivalence?

The tool above gave us a formula that looks nothing like the one we would have written by hand from the truth table, yet it is **equivalent**: it produces the exact same output for every possible input.

- Two circuits can be built very differently (different gates, different structure) and still compute the *same* function.
- Before replacing a circuit with a smaller/cheaper/faster one, engineers must **prove the two are equivalent** — this is called *equivalence checking*, a real formal-verification technique used in the chip industry to validate that an optimized circuit still matches its specification.
- The same idea applies to software: a compiler is only allowed to optimize your code if the optimized version is *equivalent* to the original program (same output, for every input).

# Why do we care about tautologies and contradictions?

- A formula (circuit) that is a **tautology** always outputs 1, no matter the inputs.
  - If you designed a circuit meant to *check* something (e.g., "is it safe to proceed?") and it turns out to be a tautology, the check is broken: it always says "yes". The check carries no information and can be deleted, saving gates, area, and power.
- A formula (circuit) that is a **contradiction** always outputs 0, no matter the inputs.
  - If you designed a circuit meant to *trigger* something (e.g., an alarm, an interrupt, an error signal) and it turns out to be a contradiction, that signal can never fire: it is dead logic that wastes silicon.

**NOTE:** In software, the same reasoning finds **dead code**: a branch guarded by a condition that is a contradiction can never execute, and a branch guarded by a tautology always executes, so the "if" is useless. Compilers and static analyzers use exactly this kind of Boolean reasoning to warn about (or remove) such code.

# Implementing real-world components

For instance, we see how to implement two common components:

1. **Half Adder**: a circuit that implements the addition of two binary numbers.
2. **Full Adder**: a circuit that implements the addition of two binary numbers with a carry input.

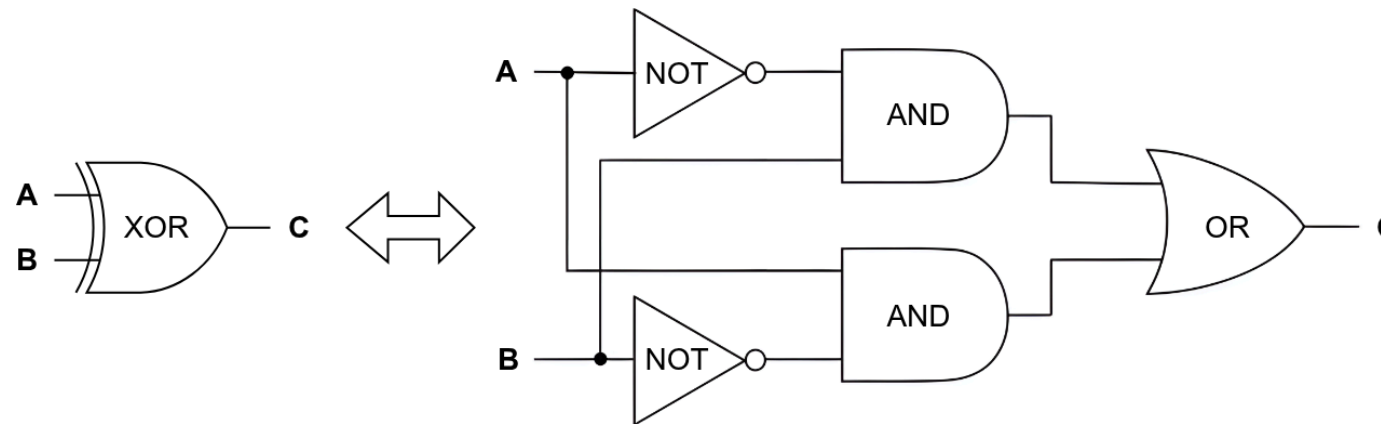
These components can be found on any computer, and are used to perform the addition of two binary numbers.

# XOR gate

To easily implement an adder, we need to implement an XOR gate:

| A | B | A XOR B |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 0       |

XOR can be implemented by combining AND, OR, and NOT gates:  $OR(AND(A, NOT(B)), AND(NOT(A), B))$



# Half Adder: boolean function

We want to design a circuit that:

- computes the sum of two binary numbers A and B
- the output is:
  - SUM(A, B): the sum of A and B
  - CARRY(A, B): the carry of SUM(A, B)

| A | B | SUM(A, B) | CARRY(A, B) |
|---|---|-----------|-------------|
| 0 | 0 | 0         | 0           |
| 0 | 1 | 1         | 0           |
| 1 | 0 | 1         | 0           |
| 1 | 1 | 0         | 1           |

# Half Adder: boolean function

We want to design a circuit that:

- computes the sum of two binary numbers A and B
- the output is:
  - $SUM(A, B)$ : the sum of A and B
  - $CARRY(A, B)$ : the carry of  $SUM(A, B)$

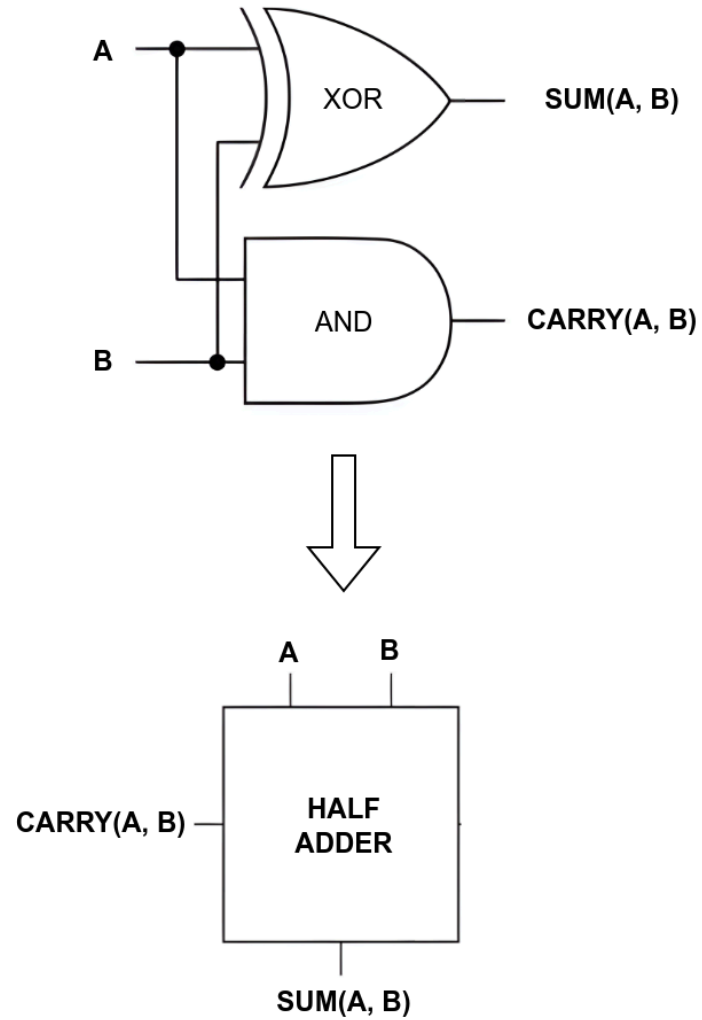
| A | B | $SUM(A, B)$ | $CARRY(A, B)$ |
|---|---|-------------|---------------|
| 0 | 0 | 0           | 0             |
| 0 | 1 | 1           | 0             |
| 1 | 0 | 1           | 0             |
| 1 | 1 | 0           | 1             |

We can easily observe by looking at the table above that:

- $SUM(A, B) = A \text{ XOR } B$
- $CARRY(A, B) = A \text{ AND } B$

**NOTE:** The formula has two outputs because when both A and B are 1, a single output bit would not be enough to represent the sum, which would be  $1_2 + 1_2 = 10_2$ ; hence, the output is split into two bits: the sum and the carry.

# Half Adder: circuit implementation

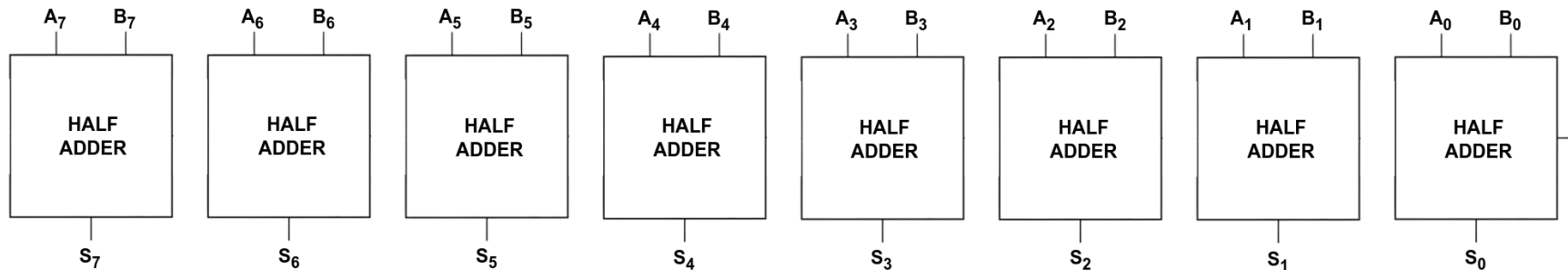


# Half Adder: cannot be chained to sum bytes

In the real world, we would like to sum two binary numbers  $A$  and  $B$  that are more than one bit long. For instance:

$$A = 100011100_2 \quad B = 101010111_2 \quad A + B = ?$$

One strategy could be to split the numbers into bits and sum them pair by pair using half adders:



where  $A_i$ ,  $B_i$ , and  $S_i$  are the  $i$ -th bit of  $A$ ,  $B$ , and  $S$  (the sum) respectively.

**PROBLEM:** We are ignoring the carry bit of each operation! The result is wrong!

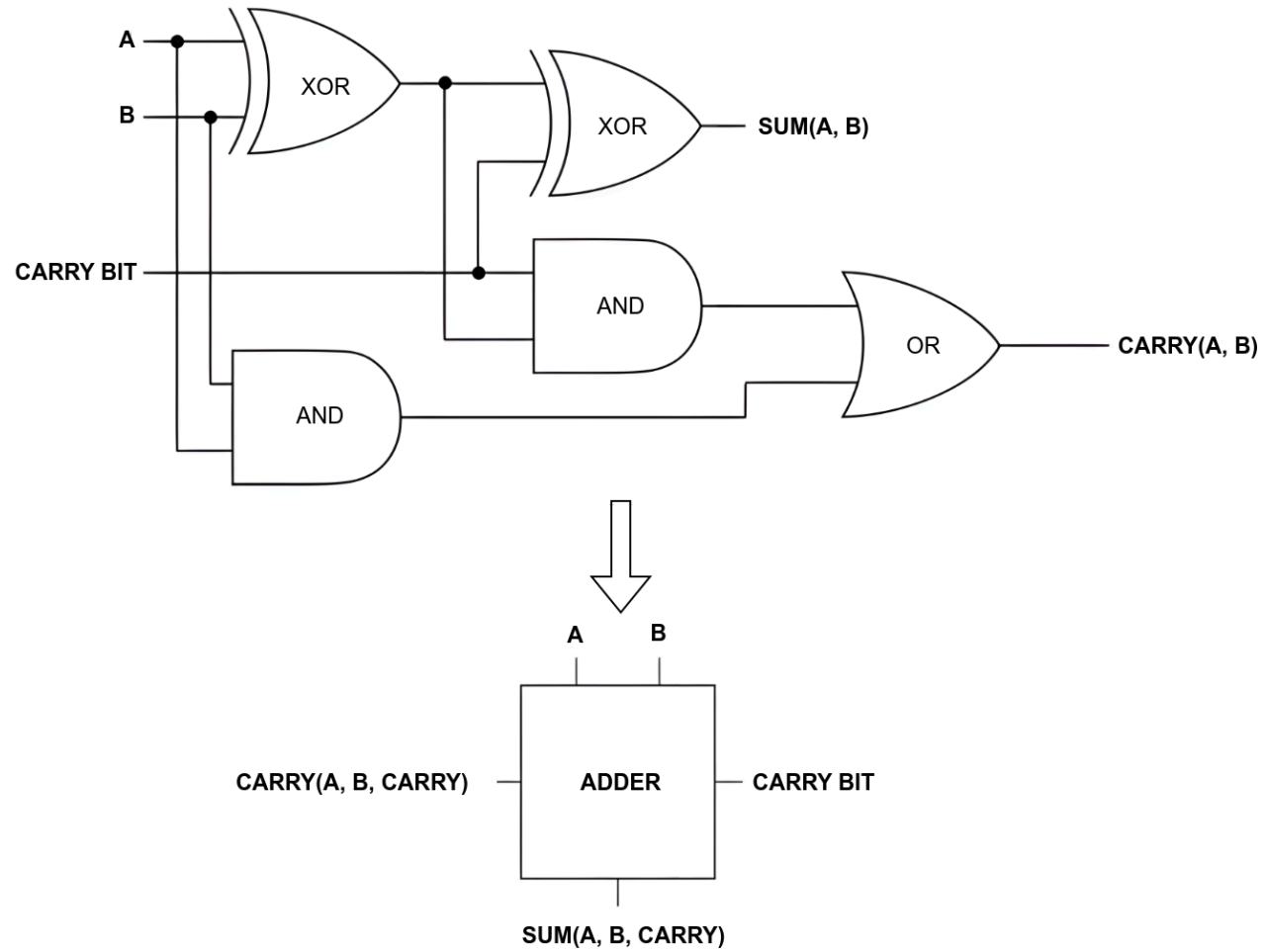
# Full Adder: boolean function

We want to design a circuit that:

- computes the sum of:
  - two bits A and B
  - a carry bit (from the previous addition)
- the output is:
  - SUM(A, B, CARRY): the sum of A, B, and CARRY
  - CARRY(A, B, CARRY): the carry of SUM(A, B, CARRY)

| A | B | CARRY | SUM(A, B, CARRY) | CARRY(A, B, CARRY) |
|---|---|-------|------------------|--------------------|
| 0 | 0 | 0     | 0                | 0                  |
| 0 | 0 | 1     | 1                | 0                  |
| 0 | 1 | 0     | 1                | 0                  |
| 0 | 1 | 1     | 0                | 1                  |
| 1 | 0 | 0     | 1                | 0                  |
| 1 | 0 | 1     | 0                | 1                  |
| 1 | 1 | 0     | 0                | 1                  |
| 1 | 1 | 1     | 1                | 1                  |

# Full Adder: circuit implementation

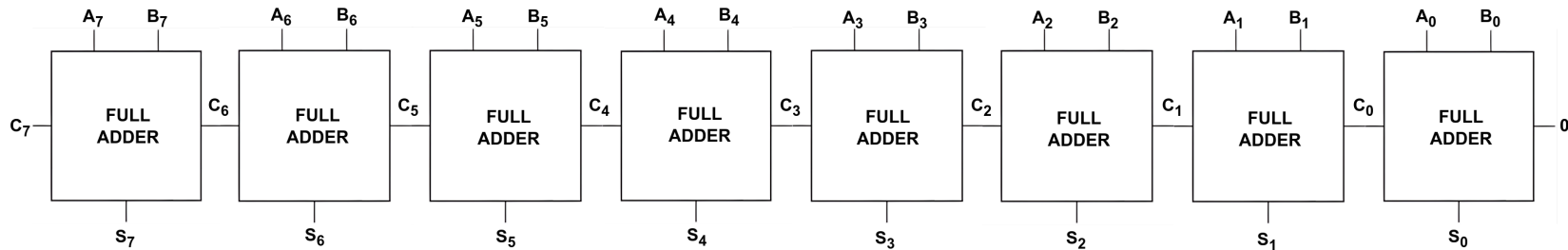


# Full Adder: summing two bytes

We now sum two bytes  $A$  and  $B$ , such as:

$$A = 100011100_2 \quad B = 101010111_2 \quad A + B = ?$$

We split the numbers into bits and sum them pair by pair using different full adders:



where  $A_i$ ,  $B_i$ , and  $S_i$  are the  $i$ -th bit of  $A$ ,  $B$ , and  $S$  (the sum) respectively. The first input carry is set to 0, and the last output carry is ignored.

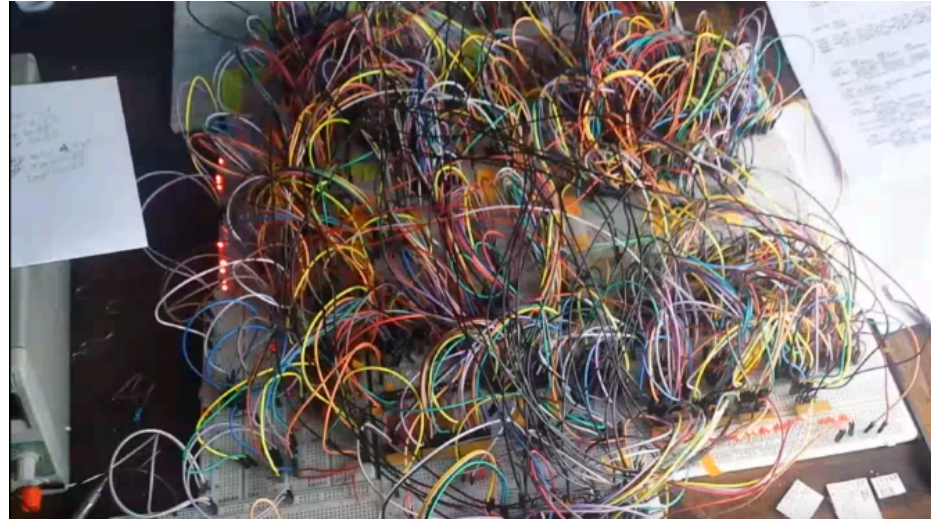
# Next: we will implement a full 8-bit computer!

We will use starter kit electronics. Spoiler of the result:



# Next: we will implement a full 8-bit computer!

We will use starter kit electronics. Spoiler of the result:

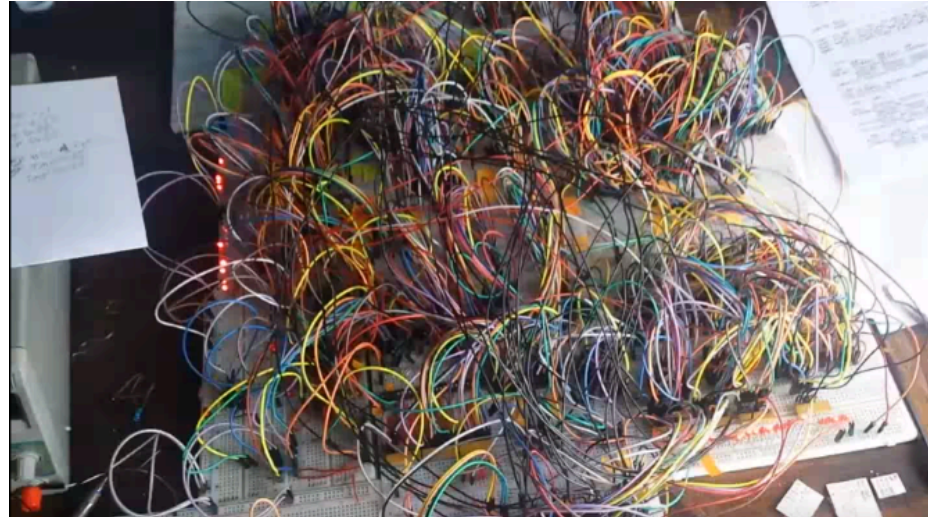


What you need: 7899 transistors, 12786 jump wires, ...



# Next: we will implement a full 8-bit computer!

We will use starter kit electronics. Spoiler of the result:



What you need: 7899 transistors, 12786 jump wires, ...

**...just kidding :)**

**NOTE:** someone has actually done it: see [here]([https://www.youtube.com/watch?v=g\\_ZaioqF1B0&ab\\_channel=PauloConstantino](https://www.youtube.com/watch?v=g_ZaioqF1B0&ab_channel=PauloConstantino)) and [here]

([https://www.youtube.com/watch?v=HyznrdDSSGM&list=PLowKtXNTBypGqImE405J2565dvjafglHU&ab\\_channel=BenEater](https://www.youtube.com/watch?v=HyznrdDSSGM&list=PLowKtXNTBypGqImE405J2565dvjafglHU&ab_channel=BenEater)).

# How are today's computers built?

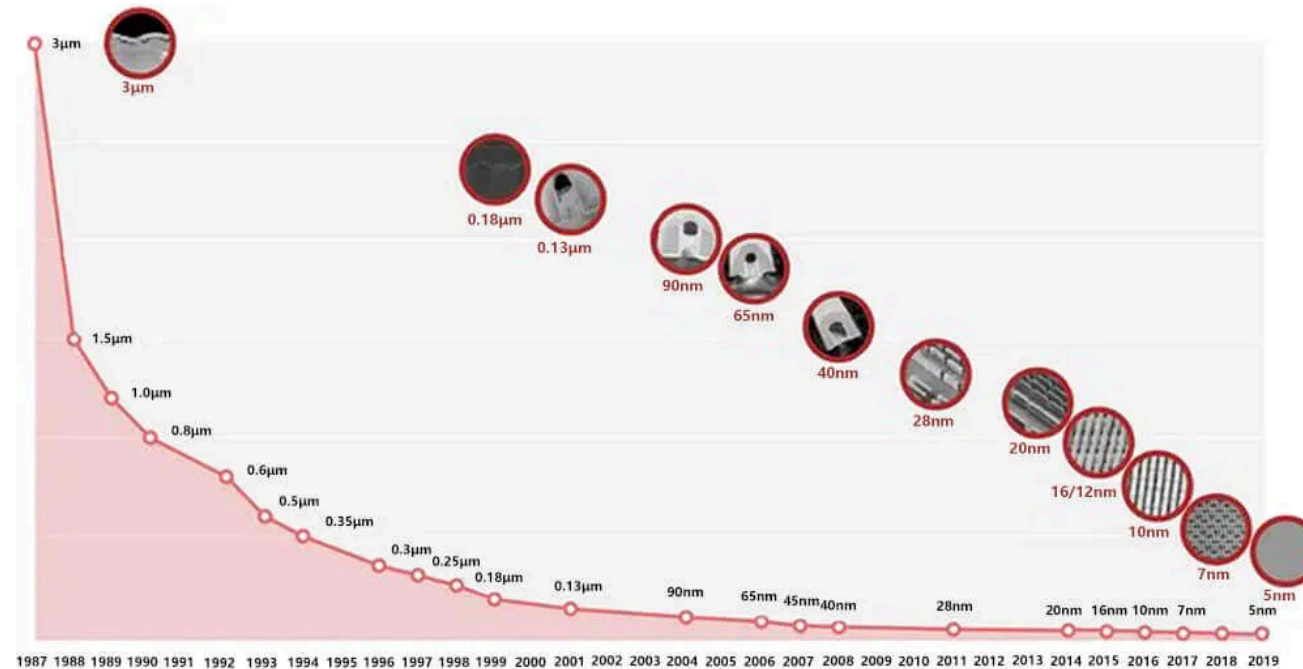
- They are based on the same ideas presented in our lectures...
- ...manufacturers are implementing millions of boolean operations
- ...to do that they need billions of gates
- ...since gates are mostly implemented using transistors
- ...they need **billions of transistors**
- ...which have to be packed in a limited surface to minimize signal propagation

They cannot use standard electronic components. They must reduce their size:

| Acronym ↕ | Name ↕                               | Year ↕ | Transistor count <sup>[91]</sup> ↕ | Logic gates number <sup>[92]</sup> ↕ |
|-----------|--------------------------------------|--------|------------------------------------|--------------------------------------|
| SSI       | <i>small-scale integration</i>       | 1964   | 1 to 10                            | 1 to 12                              |
| MSI       | <i>medium-scale integration</i>      | 1968   | 10 to 500                          | 13 to 99                             |
| LSI       | <i>large-scale integration</i>       | 1971   | 500 to 20 000                      | 100 to 9999                          |
| VLSI      | <i>very large-scale integration</i>  | 1980   | 20 000 to 1 000 000                | 10 000 to 99 999                     |
| ULSI      | <i>ultra-large-scale integration</i> | 1984   | 1 000 000 and more                 | 100 000 and more                     |

In the last 20 years, they have been able to pack billions of transistors on a chip.

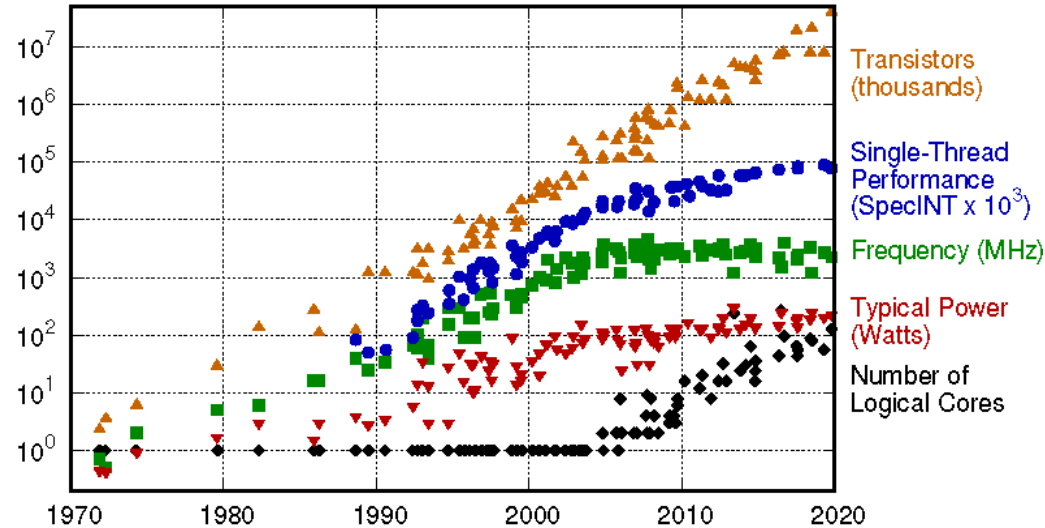
# Transistor size evolution



Only a few companies can produce transistors smaller than 10 nm:

- TSMC is leading the market: used by AMD, Apple, Samsung, Huawei, etc.
- Intel is struggling to scale under 10 nm transistors
- Samsung is also struggling to scale under 10 nm transistors

# Moore's Law



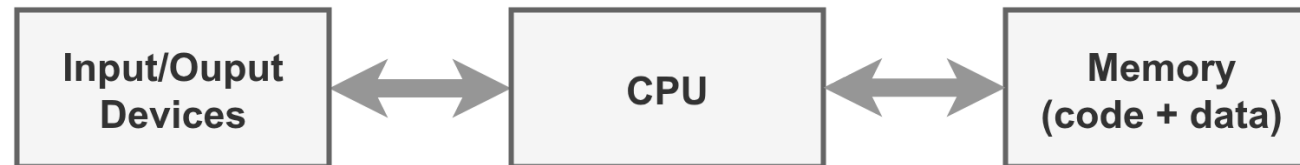
Gordon Moore (Intel): *the number of transistors in an integrated circuit doubles about every two years.*

Before 2010, more transistors translated into faster CPUs. Software was getting faster without doing anything... it was enough to buy a new CPU every year!

After 2010, more transistors translated into more cores or more cache. Unfortunately, the **free lunch is over**. software is not getting faster without doing anything... software must be re-engineered to take advantage of more cores or more cache.

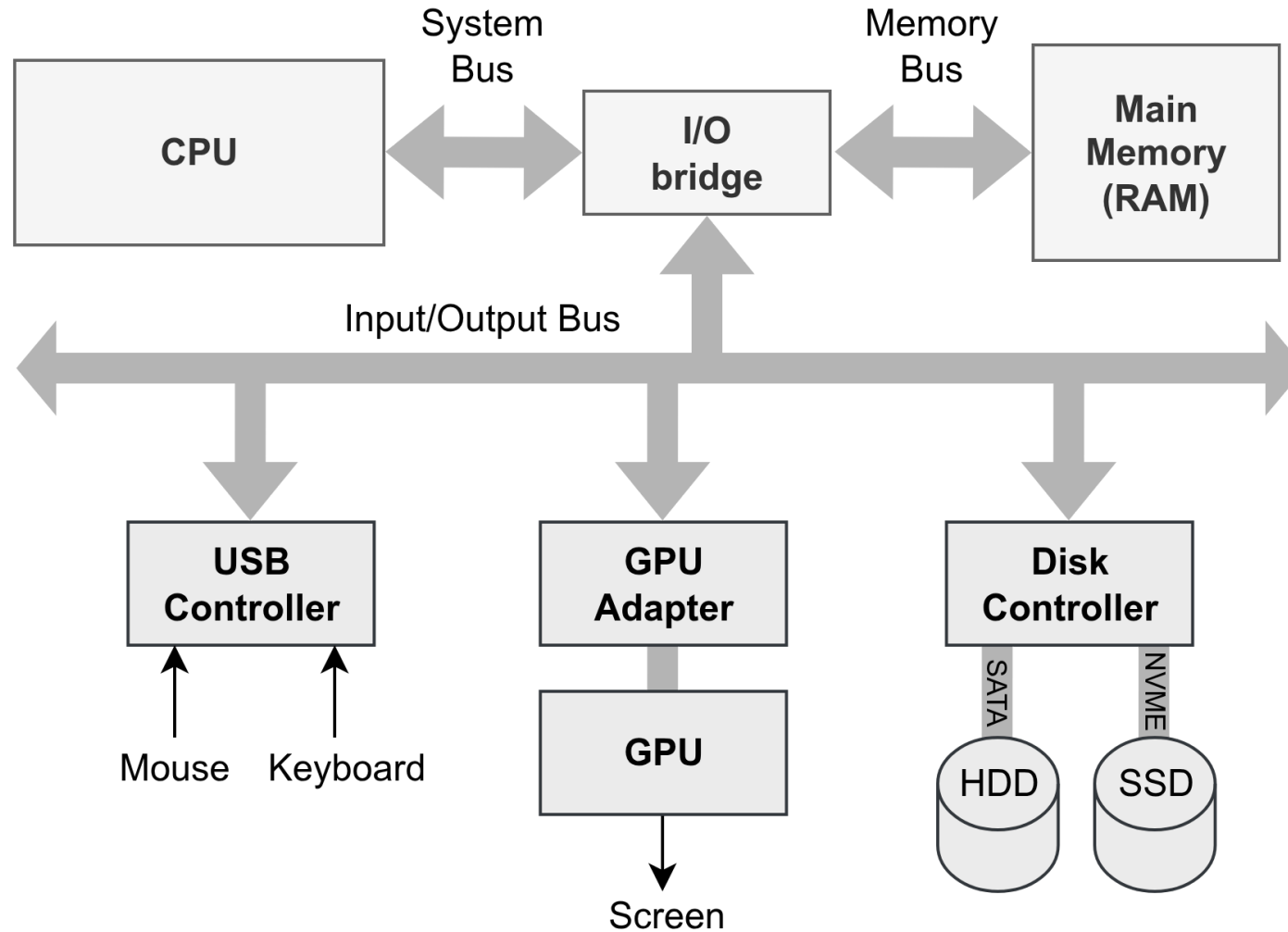
# What is inside a computer?

A (still accurate) high-level overview of a computer:

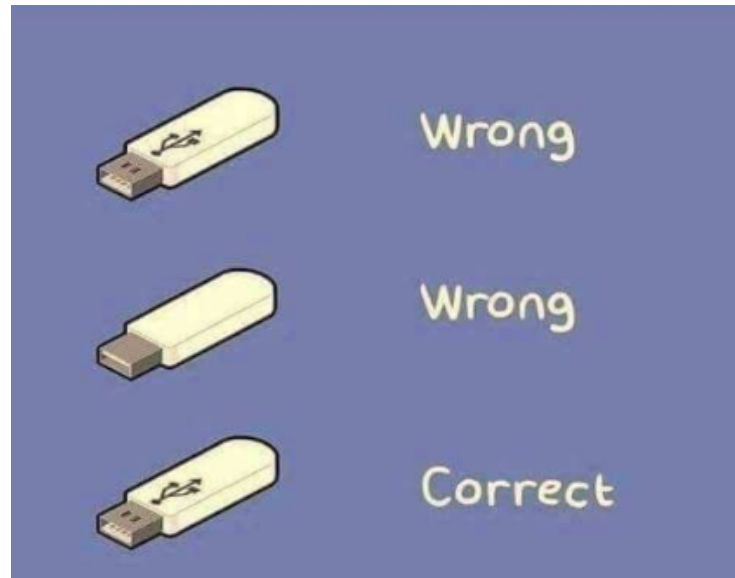


von Neumann architecture (1945)

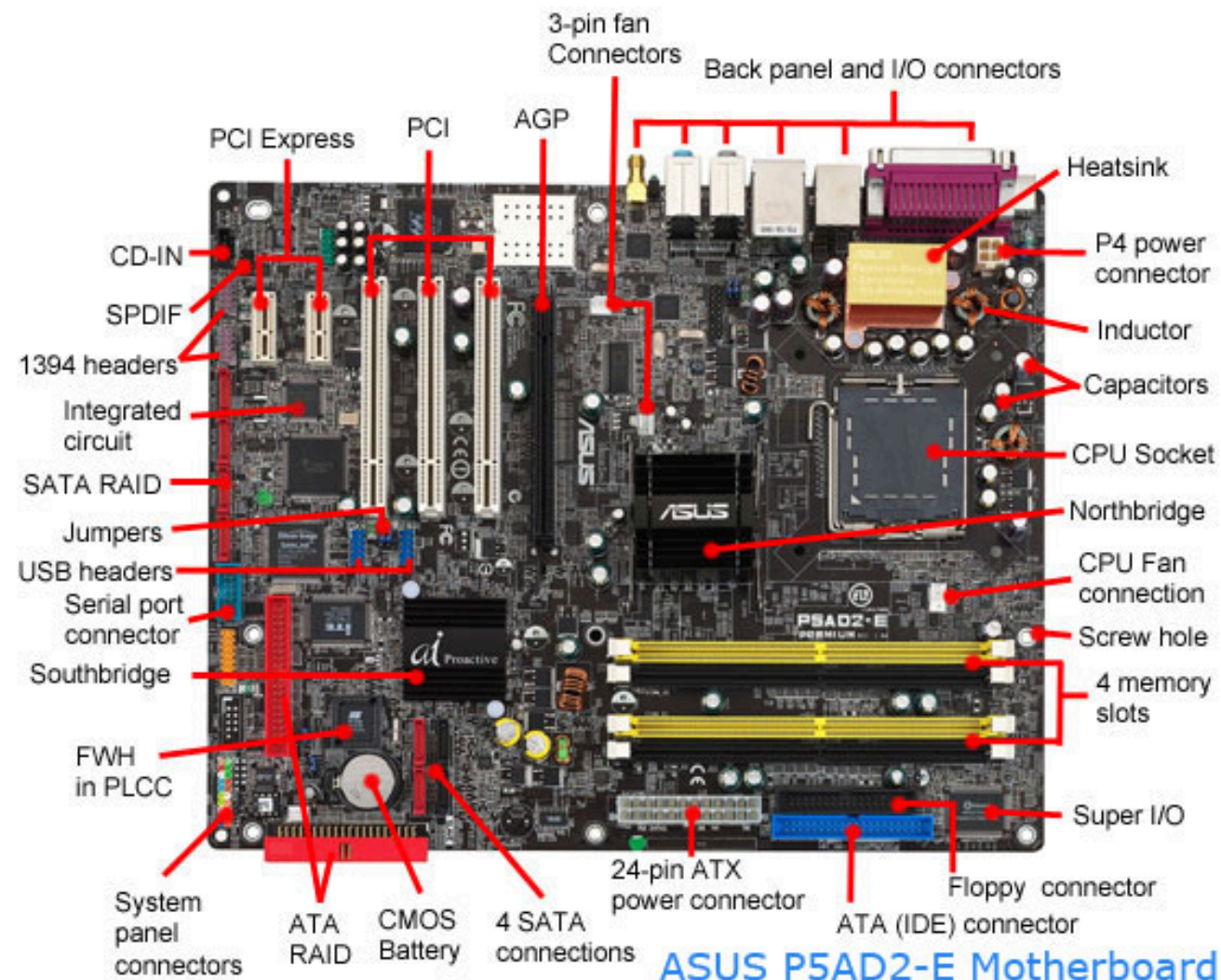
# A more detailed overview of a computer



We all know the hardest part...



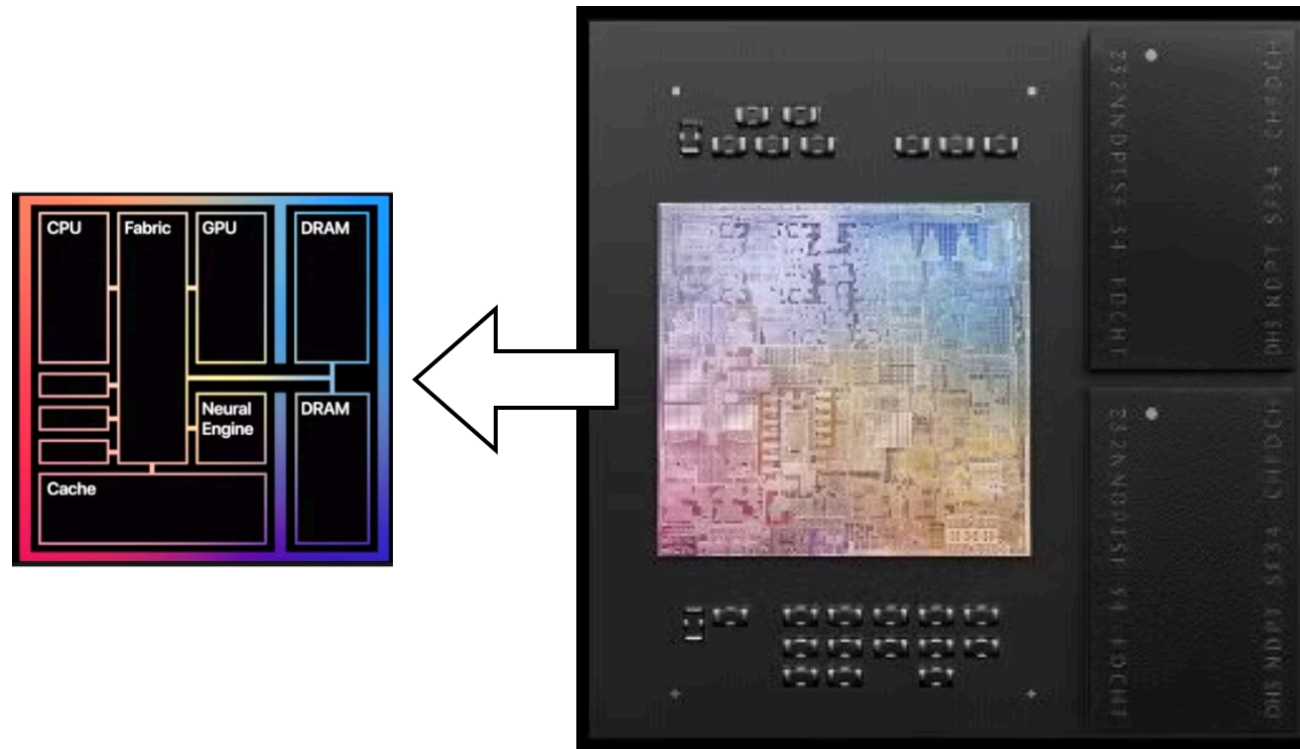
Motherboard of a computer



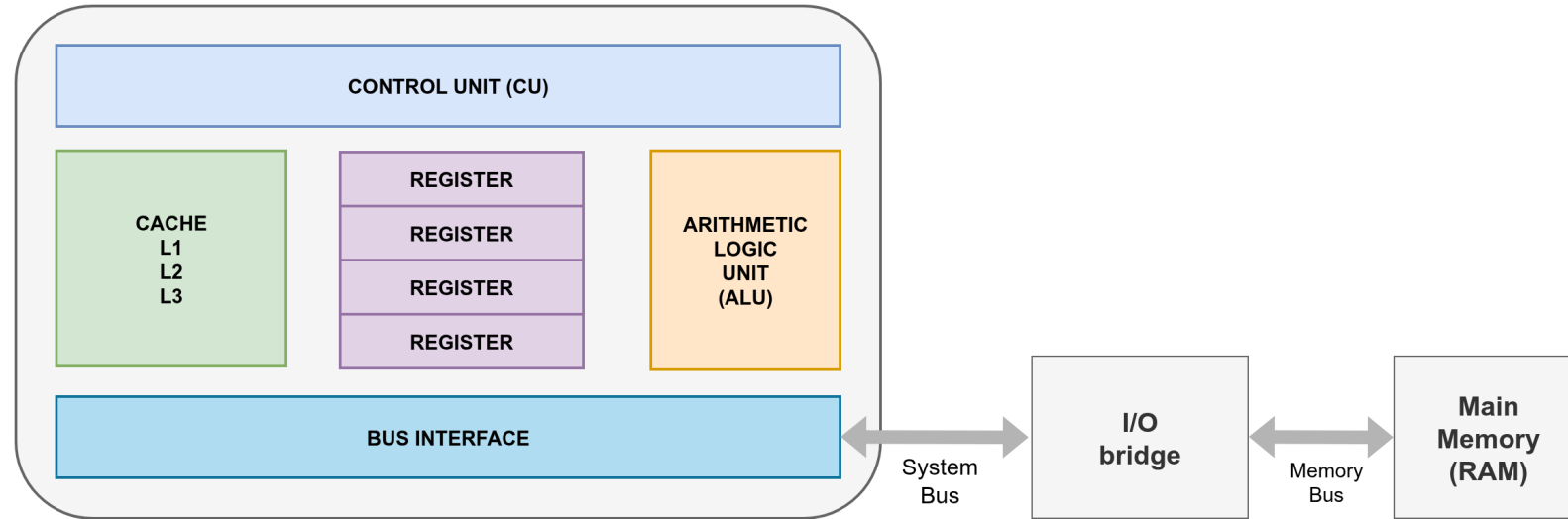
ComputerHope.com

# Modern System On Chip (SoC)

Modern SoCs integrate the CPU, GPU, storage, and other components on a single chip:



# CPU (Central Processing Unit)



- The **Arithmetic Logic Unit (ALU)** performs arithmetic operations (such as addition and subtraction) and logical operations (like AND, OR, NOT) on binary data.
- The **Control Unit** coordinates the operations of the CPU, managing the flow of data and instructions (e.g., determining the next instruction to execute, performing the fetch-decode-execute cycle).
- The **Registers** are a few (less than 10-20) small (up to 64 bits wide), fast (<0.5 nanosecond) volatile memory locations that store data and instructions for processing. Most CPUs have to first load data from RAM (~100 nanoseconds) into registers before processing it with the ALU.
- The **Cache** is a high-speed (1-16 nanoseconds) but still limited (e.g., 1-16 MB) volatile memory that stores frequently used data to speed up access.

# RAM (**R**andom **A**ccess **M**emory)

- The main (volatile) memory of the computer, where data and instructions are stored for processing before being loaded into the CPU.
- Implemented with Dynamic Random Access Memory (DRAM) chips, based on capacitors (require regular refreshes to maintain data).
- The capacity of RAM is nowadays on the order of gigabytes (GB) or terabytes (TB).
- Much faster ( $1000 - 100000\times$ ) than secondary storage (e.g., hard drives, SSDs) but significantly slower ( $10 - 100\times$ ) than CPU registers.
- Like a huge array of consecutive bytes, each identified by its address (e.g.,  $0, 1, 2, \dots, 2^{64} - 1$ ).

# Disks

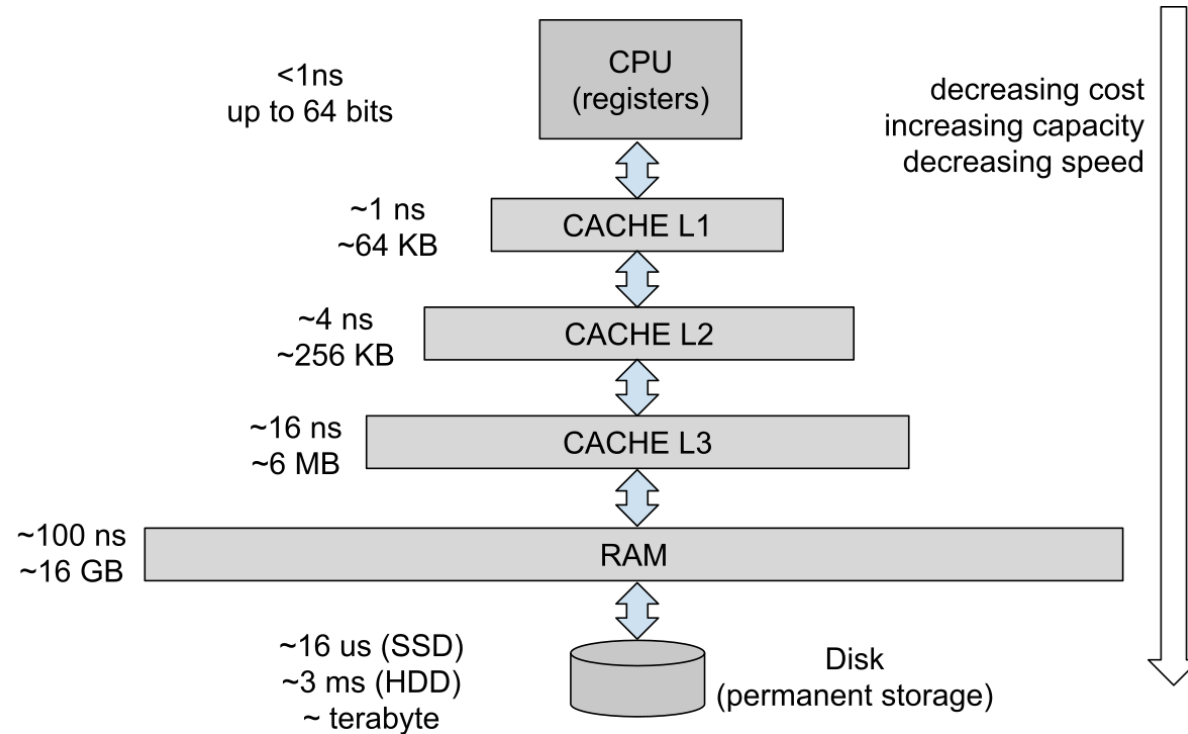
Permanent storage devices used to store data and programs for long-term use:

- **Hard Drives (HDDs)**: use spinning disks to store data, quite slow speed ( $\sim 200$  MB/s) but large capacity (up to 20 TB).
- **Solid-state disks (SSDs)**: use flash memory to store data, much faster speed ( $\sim 1$ -4 GB/s) but smaller capacity (up to 2 TB).

Disks can be connected to the computer in different ways:

- USB (Universal Serial Bus): used for external disks.
- IDE (Integrated Drive Electronics): used for mechanical disks.
- SATA (Serial Advanced Technology Attachment): used for mechanical disks and older SSDs.
- NVMe (Non-Volatile Memory Express): used for modern SSDs.

# Memory hierarchy



Fast memory is expensive, hence a complex hierarchy is used to compromise between cost and performance: larger data is stored in slower, cheaper memory, that, when needed, is loaded into faster, more expensive memory into smaller amounts.

Cloud...?



# How can we program a computer?

In the next lecture, we will cover the software side of the story.

